MCT - 454L Mobile Robotics

Lab Manual 7: Autonomous Navigation with Nav2 and Multi-Waypoint Mission Planning

Objective

This lab session builds upon Lab Manual 5 by enabling students to perform autonomous navigation using a pre-built map, the Nav2 navigation stack, and the TurtleBot3 platform in a Gazebo simulation. Students will load a previously saved map, localize the robot using AMCL, and plan multi-waypoint missions using both the Nav2 command-line interface and a custom Python node. By the end of this session, students will understand the full autonomous navigation pipeline in ROS 2.

Prerequisites

- Completion of Lab Manual 5 (Gazebo, RViz, SLAM with Cartographer)
- A saved map from Lab Manual 5 (my_map.pgm and my_map.yaml) in ~/maps/
- TurtleBot3 and Nav2 packages installed
- ROS 2 Humble environment sourced
- Basic familiarity with Python and ROS 2 nodes

Background

Nav2 Navigation Stack

> **What is Nav2?**
> Nav2 (Navigation2) is the standard autonomous navigation framework for ROS 2. It provides a suite of servers for path planning, trajectory following, behavior trees, and map-based localization.

The Nav2 stack consists of the following key components:

- BT Navigator: Executes behavior trees to coordinate navigation tasks.
- Planner Server: Computes a global path from the robot's current pose to a goal (default: NavFn / Smac Planner).
- Controller Server: Follows the computed path using a local trajectory controller (default: DWB controller).
- Smoother Server: Smooths global paths for more natural robot motion.
- Recoveries / Behavior Server: Handles failure cases (spin, backup, wait).

- Map Server: Loads and serves a static map (PGM + YAML) to the stack.
- AMCL: Adaptive Monte Carlo Localization — estimates robot pose on the loaded map using LiDAR scans.
- Waypoint Follower: Executes a sequence of navigation goals (waypoints) in order.

AMCL Localization

> **What is AMCL?**
> Adaptive Monte Carlo Localization (AMCL) uses a particle filter to estimate the robot's position and orientation on a known map. It subscribes to LiDAR scan data and the odometry topic, and publishes the estimated pose. AMCL requires a good initial pose estimate, which is set using the 2D Pose Estimate tool in RViz.

Map File Format

The map saved in Lab 5 consists of two files:

- my_map.pgm — A grayscale Portable Graymap image where white = free space, black = obstacle, grey = unknown.
- my_map.yaml — A metadata file specifying resolution, origin, and thresholds for occupancy interpretation.

Package Installation

Install the Nav2 packages if not already installed:

```
sudo apt update
sudo apt install ros-humble-navigation2 ros-humble-nav2-bringup
sudo apt install ros-humble-turtlebot3-navigation2
```

Environment Setup

Ensure the following environment variables are set in every terminal (or in ~/.bashrc):

```
source /opt/ros/humble/setup.bash
export TURTLEBOT3_MODEL=burger
```

> **Note:** Add both lines to ~/.bashrc to avoid re-exporting in each new terminal session.

Step 1: Launch Gazebo Simulation

Launch the same TurtleBot3 world used in Lab Manual 5 so that the robot starts in the same environment as the saved map:

```
ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py
```

Wait for Gazebo to fully load. Verify that the TurtleBot3 model appears in the world. Use rqt_graph to observe running nodes:

```
rqt_graph
```

Note: The robot's starting position in Gazebo must match the coordinate frame of your saved map. If you see large localization errors, restart Gazebo and ensure the robot spawns at (0, 0, 0).

Step 2: Launch Nav2 with Saved Map

Open a new terminal and launch the TurtleBot3 Nav2 bringup with the saved map:

```
ros2 launch turtlebot3_navigation2 navigation2.launch.py \
  use_sim_time:=True \
  map:=$HOME/maps/my_map.yaml
```

This command will:

- Launch the Map Server to load and publish my_map.yaml.
- Start AMCL for particle-filter based localization.
- Launch the Planner Server, Controller Server, and BT Navigator.
- Open RViz pre-configured with Nav2 visualization panels.

Note: If RViz does not open automatically, launch it manually: ros2 launch nav2_bringup rviz_launch.py

Step 3: Set Initial Robot Pose in RViz

AMCL needs an initial pose estimate to begin localizing. The particle cloud will be spread randomly until you set this:

1. In RViz, click the 2D Pose Estimate button in the toolbar.
2. Click on the map at the approximate location where the robot started in Gazebo (typically near the origin).
3. Drag the arrow in the direction the robot is facing (usually along the positive X axis).
4. Observe the particle cloud (small red arrows) converging around the estimated pose.
5. Drive the robot slightly using teleoperation to help AMCL refine its localization:

   ```
   ros2 run turtlebot3_teleop teleop_keyboard
   ```

6. Confirm the laser scan (shown in RViz) aligns with the map walls. If misaligned, re-set the 2D Pose Estimate.

Note: Good localization is critical before sending navigation goals. A poorly set initial pose will result in the robot colliding with obstacles or planning invalid paths.

Step 4: Send a Single Navigation Goal

Using RViz

1. In RViz, click the Nav2 Goal button (or 2D Nav Goal in older configs).
2. Click a free space on the map and drag to set the goal orientation.
3. Observe Nav2 compute a global path (shown as a colored line) and the robot begin moving.
4. Watch the local costmap updating in real time around the robot.

Using the Command Line

Goals can also be sent programmatically via the action interface:

```
ros2 action send_goal /navigate_to_pose nav2_msgs/action/NavigateToPose
\
  "{pose: {header: {frame_id: 'map'}, pose: {position: {x: 1.0, y: 0.5,
z: 0.0},
  orientation: {x: 0.0, y: 0.0, z: 0.0, w: 1.0}}}}"
```

Step 5: Multi-Waypoint Navigation Using Nav2 Waypoint Follower

> **Nav2 Waypoint Follower**
> Nav2 includes a FollowWaypoints action server that accepts a list of poses and navigates through each in sequence. The robot pauses briefly at each waypoint before proceeding to the next. This is useful for patrol routes, inspection missions, and coverage tasks.

Writing the Waypoint Navigation Python Node

Create a new Python file for the waypoint navigation node:

```
mkdir -p ~/ros2_ws/src/lab6_nav/lab6_nav
cd ~/ros2_ws/src/lab6_nav/lab6_nav
nano waypoint_navigator.py
```

Copy the following starter code into the file:

```
import rclpy
from rclpy.node import Node
from rclpy.action import ActionClient
from nav2_msgs.action import FollowWaypoints
from geometry_msgs.msg import PoseStamped
import time
```

```
class WaypointNavigator(Node):
    def __init__(self):
        super().__init__('waypoint_navigator')
        self._client = ActionClient(self, FollowWaypoints,
'follow_waypoints')

    def send_waypoints(self, waypoints):
        self.get_logger().info('Waiting for FollowWaypoints action
server...')
        self._client.wait_for_server()
        goal_msg = FollowWaypoints.Goal()
        goal_msg.poses = waypoints
        self.get_logger().info(f'Sending {len(waypoints)}
waypoints...')
        send_goal_future = self._client.send_goal_async(goal_msg)
        rclpy.spin_until_future_complete(self, send_goal_future)
        goal_handle = send_goal_future.result()
        if not goal_handle.accepted:
            self.get_logger().error('Goal rejected by server!')
            return
        self.get_logger().info('Goal accepted. Navigating...')
        result_future = goal_handle.get_result_async()
        rclpy.spin_until_future_complete(self, result_future)
        self.get_logger().info('All waypoints reached!')

def make_pose(x, y, yaw_w):
    pose = PoseStamped()
    pose.header.frame_id = 'map'
    pose.pose.position.x = x
    pose.pose.position.y = y
    pose.pose.position.z = 0.0
    pose.pose.orientation.z = yaw_w
    pose.pose.orientation.w = 1.0
    return pose

def main(args=None):
    rclpy.init(args=args)
    navigator = WaypointNavigator()

    # TODO: Define at least 5 waypoints within your map
    waypoints = [
        make_pose(0.5,  0.0,  1.0),   # Waypoint 1
        make_pose(1.0,  0.5,  1.0),   # Waypoint 2
        make_pose(1.0, -0.5,  1.0),   # Waypoint 3
```

```
            make_pose(0.0, -0.5,  0.0),    # Waypoint 4
            make_pose(0.0,  0.0,  1.0),    # Waypoint 5 (return to origin)
        ]

        navigator.send_waypoints(waypoints)
        navigator.destroy_node()
        rclpy.shutdown()

    if __name__ == '__main__':
        main()
```

Running the Waypoint Node

Run the node directly (ensure Nav2 and Gazebo are already running):

```
python3 waypoint_navigator.py
```

Observe the robot navigating through each waypoint in RViz. The global path will update at each waypoint as a new segment is planned.

Step 6: Visualizing Navigation Data in RViz

Ensure the following display plugins are active in RViz during navigation:

- Map — Displays the loaded occupancy grid from the Map Server.
- Global Costmap — Overlays inflation layers and obstacle data used by the global planner.
- Local Costmap — Shows the rolling local costmap used by the DWB controller.
- Global Path — Visualizes the planned global trajectory (topic: /plan).
- Local Path — Shows the local trajectory being followed (topic: /local_plan).
- AMCL Particle Cloud — Displays localization uncertainty (topic: /particle_cloud).
- TF — Shows coordinate frame relationships (map -> odom -> base_footprint).
- Odometry — Displays real-time odometry arrows (topic: /odom).
- LaserScan — Shows live LiDAR data for localization validation.

> **Note:** Use Fixed Frame: map in RViz Global Options to align all data in the world reference frame.

Tasks for Students

Task 1: Single Goal Navigation

1. Send at least 3 different single navigation goals using the RViz Nav2 Goal button.

2. For each goal, record the start and end pose, and note whether the robot succeeded or recovered from an obstacle.
3. Use ros2 topic echo /amcl_pose to inspect the robot's estimated pose before and after each goal.

Task 2: Define and Execute a Multi-Waypoint Mission

Fill in the table below with at least 5 waypoints within your map. Refer to the coordinate grid in RViz or use ros2 topic echo /amcl_pose to identify valid coordinates.

Waypoint #	X (m)	Y (m)	Z (m)	Yaw (rad)	Description
1			0.0		
2			0.0		
3			0.0		
4			0.0		
5			0.0		

1. Modify the waypoint_navigator.py node to use your 5 defined waypoints.
2. Run the node and observe the robot complete the full mission.
3. Take screenshots of RViz at each waypoint showing the robot's position and the path taken.

Task 3: Dynamic Waypoint Injection

Extend your waypoint_navigator.py to accept waypoints from the command line as arguments instead of hardcoding them. Example usage:

```
python3 waypoint_navigator.py 0.5 0.0 1.0 1.0 0.5 1.0 0.0 0.0 1.0
```

Each group of three values represents x, y, and orientation_w for one waypoint. Parse these arguments inside main() and build the waypoints list dynamically.

Task 4: Costmap Observation

1. Navigate the robot close to an obstacle and observe how the local costmap inflates around it.
2. Examine the topic list and identify the topics used by the global and local costmaps.
3. Record the costmap topic names and describe the difference between the global and local costmap roles.

Task 5: Navigation Recovery Behaviors

1. Manually place a dynamic obstacle in Gazebo (insert a box model) in the robot's planned path.
2. Observe and describe how Nav2 recovers: does it replan, spin in place, or back up?
3. Identify which recovery behavior server handles the response using rqt_graph.

Conclusion

This lab session demonstrates the complete autonomous navigation pipeline in ROS 2 using the Nav2 stack, a pre-built occupancy map, AMCL localization, and TurtleBot3. Students experience the integration of mapping (Lab 5) with navigation (Lab 7) and gain practical skills in waypoint-based mission planning, localization validation, and costmap interpretation. These skills form the foundation for more advanced autonomous behaviors including coverage planning, multi-robot coordination, and dynamic obstacle avoidance.

Deliverables

Submit the following at the end of the lab session or as instructed:

1. A short report detailing all steps followed and observations made at each stage.
2. Completed waypoint table (Task 2) with all 5 waypoint coordinates filled in.
3. Python source code for waypoint_navigator.py (Task 2) and the dynamic argument version (Task 3).
4. Screenshots of RViz showing: loaded map with AMCL particle cloud, global and local paths during multi-waypoint navigation, robot pose at each waypoint, and the active costmaps.
5. Screenshots of rqt_graph showing running Nav2 nodes and their topic connections.
6. Written observations on recovery behavior encountered in Task 7.
7. Conclusion summarizing the learning experience, challenges faced, and comparison between SLAM (Lab 5) and navigation (Lab 7) workflows.